Hafta 2: İteratörler ve STL Algoritmaları

Yazılım Geliştirme II – Çalışma Kağıdı

Ad Soyad: _____	Numara: _____	Tarih: _____

Yönergeler

- Bu çalışma kağıdı **6 bölümden** oluşmaktadır.
- Çoktan seçmeli sorularda doğru seçeneği **daire içine alın**.
- Boşluk doldurma sorularında cevabınızı **çizgi üzerine** yazın.
- Açık uçlu sorularda cevabınızı **verilen alana** yazın.
- Süre: **40 dakika**

Bölüm A: Çoktan Seçmeli Sorular

1. `std::list`'in sağladığı iteratör kategorisi aşağıdakilerden hangisidir? **2 puan**
   A) Random Access Iterator
   B) Forward Iterator
   C) Bidirectional Iterator
   D) Input Iterator

2. `std::remove` algoritması gerçekte ne yapar? **2 puan**
   A) Elemanları kapsayıcıdan siler
   B) Eşleşen elemanları sona taşır ve yeni mantıksal son iteratörü döndürür
   C) Kapsayıcının boyutunu küçültür
   D) Elemanları sıralar ve tekrarları siler

3. `std::sort` algoritmasının ortalama zaman karmaşıklığı nedir? **2 puan**
   A) $O(n)$
   B) $O(n^2)$
   C) $O(n \log n)$
   D) $O(\log n)$

4. Lambda ifadesinde `[&]` yakalama listesi ne anlama gelir? **2 puan**
   A) Tüm yerel değişkenleri değer ile yakalar
   B) Tüm yerel değişkenleri referans ile yakalar
   C) Hiçbir değişken yakalanmaz
   D) Sadece `this` işaretçisini yakalar

5. `std::accumulate` fonksiyonuna boş bir aralık verildiğinde ne döndürür? **2 puan**
   A) 0
   B) Başlangıç değerini (init)
   C) `std::out_of_range` istisnası fırlatır
   D) Tanımsız davranış

6. Aşağıdaki algoritmalardan hangisi giriş aralığının sıralı olmasını gerektirir? **2 puan**

   A) `std::find`
   B) `std::count`
   C) `std::binary_search`
   D) `std::for_each`

7. Remove-erase deyiminin doğru kullanımı aşağıdakilerden hangisidir? **2 puan**

   A) `v.erase(std::remove(v.begin(), v.end(), x));`
   B) `v.erase(std::remove(v.begin(), v.end(), x), v.end());`
   C) `std::remove(v.begin(), v.end(), x); v.shrink_to_fit();`
   D) `v.remove(x); v.erase();`

8. `std::partition` algoritması ne döndürür? **2 puan**

   A) Bölünen eleman sayısı
   B) İkinci grubun başlangıcına işaret eden iteratör
   C) Kapsayıcının yeni boyutu
   D) Bir `bool` değeri

Bölüm B: Doğru / Yanlış

Her ifadenin doğru (D) veya yanlış (Y) olduğunu belirtin.

1. `std::sort` algoritması `std::list` iteratörleri ile çalışabilir.
2. `std::binary_search` sıralı bir aralık gerektirir.
3. Lambda ifadeleri değişkenleri hem değer hem referans ile aynı anda yakalayabilir.
4. `std::function` sıfır çalışma zamanı maliyetine sahiptir.
5. `std::stable_sort` eşit elemanların göreli sırasını korur.
6. C++20 `std::erase_if` tüm standart kapsayıcılarla çalışır.

Bölüm C: Eşleştirme

Sol sütundaki iteratör kategorilerini sağ sütundaki desteklenen işlemlerle eşleştirin.

#	İteratör Kategorisi	Cevap	#	Desteklenen İşlem
1	Input Iterator		a	İleri ve geri hareket, okuma/yazma
2	Output Iterator		b	Sadece okuma, tek geçiş
3	Forward Iterator		c	Rastgele erişim, aritmetik işlemler
4	Bidirectional Iterator		d	Sadece yazma, tek geçiş
5	Random Access Iterator		e	İleri hareket, çok geçişli okuma/yazma

Bölüm D: Boşluk Doldurma

Aşağıdaki cümlelerde boş bırakılan yerleri uygun ifadelerle doldurun.

1. STL iteratörleri, kapsayıcılar ve _____ arasında köprü görevi görür.
2. `std::sort` algoritması en az _____ kategorisinde iteratör gerektirir.
3. Lambda ifadesinin genel söz dizimi: _____ (parametreler) _____ { gövde } şeklindedir.
4. `std::remove` elemanları gerçekten silmez, sadece _____ ve yeni mantıksal sonu döndürür.
5. `std::accumulate` fonksiyonu _____ başlık dosyasında tanımlıdır.
6. `std::partition` algoritması, koşulu sağlayan elemanları aralığın _____ kısmına taşır.
7. C++20 ile gelen _____ fonksiyonu, remove-erase deyiminin yerini alarak kapsayıcıdan elemanları doğrudan siler.

Bölüm E: Kod Çıktısı

Aşağıdaki C++ kodlarının çıktılarını yazın.

1. **3 puan**

```
std::vector<int> v = {5, 3, 8, 1, 9, 2};
std::sort(v.begin(), v.end());
for (int x : v) std::cout << x << " ";
```

Çıktı:

2. **3 puan**

```
std::vector<int> v = {1, 2, 3, 4, 5, 6};
auto it = std::remove_if(v.begin(), v.end(),
    [](int x) { return x % 2 == 0; });
v.erase(it, v.end());
for (int x : v) std::cout << x << " ";
```

Çıktı:

3. **4 puan**

```
std::vector<int> v = {10, 20, 30, 40, 50};
int toplam = std::accumulate(v.begin(), v.end(), 0);
auto sayac = std::count_if(v.begin(), v.end(),
    [](int x) { return x > 25; });
std::cout << toplam << " " << sayac;
```

Çıktı:

Bölüm F: Açık Uçlu Sorular

1. Remove-erase deyimini açıklayın ve C++20 `std::erase`/`std::erase_if` ile karşılaştırın. Hangi durumlarda hangisi tercih edilir? **6 puan**

2. Aşağıdaki OBYS senaryosunu gerçekleştiren bir C++ programı yazın: **8 puan**

   - `std::vector<pair<int, double>>` ile öğrenci numarası → not ortalaması listesi oluşturun
   - `std::sort` ile listeyi not ortalamasına göre büyükten küçüğe sıralayın (lambda kullanın)
   - `std::find_if` ile not ortalaması 3.0 üzeri ilk öğrenciyi bulun (lambda kullanın)
   - Sonucu yazdırın

3. Aşağıdaki kodda bir hata vardır. Hatayı bulun ve düzeltilmiş hâlini yazın. **5 puan**

```cpp
std::vector<int> notlar = {85, 92, 78, 95, 60};
int esik = 80;

auto yuksekler = std::count_if(notlar.begin(), notlar.end(),
    [](int n) { return n >= esik; });

std::cout << "80 ve uzeri: " << yuksekler << '\n';
```

Başarılar!